Prompt Template, Output Parser, Chain

Thursday, February 26, 2026 6:04 PM

Recap:

Chat prompt template → useful ⇒ Chat models

LLM ← Single ← Reg (Legacy)

H → A → Sys, A ← H — Convers ← Chat Emb

Chat models → Chat OpenAI, Chat-GoogGenAI, ChatGroq, Chat-Mistral

3 types Messages

1) System → LLM ↑ Behavior

2) Human ✓ → Query ask

LLM → User

3) AI Message → Few shot prompt → Make LLM beh/reason way user want

Thru Langchain → To LLM

Give a Human Message

('human', ' msg ')

✓ HumanMessage(' msg ') ✓

✓ langchain.messages. import HumanMessage, SystemMes AI message

[✓

✓ ('system', ' ')

('human', ' ')

('assistant', ' ')]

[HumanMessage (✓)

(OR) SystemMessage ( ) → LLM.invoke (messages) → respond AIMessage

AIMessage ( )]

Suitable → provide → Clear Instru LLM → without any fields to be filled

If it is not

If it is not a template | without any fields to be filled.

Template

add some fields to it

appin → Students → clear queries

Students queries → Concisely explain what is ML in 3 sentences

Template

User needs to prov info about

[Concisely explain what is {topic} in {number of sentences}]

topic: [ ] ✓

number: [ ] ✓

Response

To input Templates with fields (empty) to an LLM

prev syntax

✗ (sys / hum / ai , msg ,) | human / sys / AI Msg (msg)

Input is a Template ✗ messages: [( ), ( ), ( )] → LLM.invoke(messages)

[sys, hum, ai msg] ✗ ✗

Chat Prompt Template:

Sys: explain any topic in 3 sentences

hum: Explain ML ?

✓ Sys_temp: Respond the user query in { num } of sentences.

Field

hum_temp: Explain { topic } briefly?

Step-1 : Message prompt Temp. → Import → langchain_core.prompts

To create a sys_msg with sys_temp.

import SysMPT
HMPT
AMPT

sys_msg =
SystemMessagePromptTemplate.from_template
(sys_temp.)

Create hum_temp → hum_msg

hum_msg =
HumanMessagePromptTemplate.from_template
(hum_temp.)

X X messages = [sys_msg, human_msg] X X

To Combine sys, human msg as a prompt and give the prompt to LLM as input

↓

follow another syntax

from langchain_core.prompts import ChatPromptTemplate

Combine the message

messages: ChatPromptTemplate.from_messages
([sys_msg, hum_msg.])

→ No limit

Created from Templates

([sm1, hm1, ai1, sm2, hm2, ai2, ----])

Give messages to LLM as input

Chain = messages | LLM ✓

Give messages to LLM as input

Chain = messages | LLM ✓

response = Chain.invoke({ key: value })

{ 'topic': 'ML', 'number': 8 }.

clean output = response.content

↳ clean o/p.

Chain · Syntax

Chat Prompt Template | LLM | Parser [| LLM | parser]

Few Shot Chat message prompt Template

Create sys_temp.

hum_temp.

ai_temp. ✓

Conv to messages from temp.

Combine Messages → Chat Prompt Template ✓

from langchain.prompts import Few Shot Chat Mess Promp Temp

few_shot = FSCMPT (examples = ✓

example_prompt =

input var = [ ]).

examples: [{ K:V } ✓

i/p: values, o/p: values]

⇒ Output parser < Str, Json

parser = StrOutputPar()

Chain : pt | LLM | parser